```
import os
import random
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D,
Flatten
from PIL import Image
from sklearn.metrics import classification_report

print('TensorFlow version:', tf.__version__)
```

```
TensorFlow version: 2.19.0
```

!unzip "/content/drive/MyDrive/level6/ai/week5/FruitinAmazon.zip" -d "/content/drive/MyDrive/level6/ai/week6"

```
USE_AMAZON = True                                    # ← Change to True
when you have the dataset
TRAIN_DIR     =
'/content/drive/MyDrive/level6/ai/week6/FruitinAmazon/train'
# ← Update to your Train/ folder path
TEST_DIR      =
'/content/drive/MyDrive/level6/ai/week6/FruitinAmazon/test'
# ← Update to your Test/  folder path

# IMG_HEIGHT      = 128
# IMG_WIDTH       = 128
# BATCH_SIZE      = 16
# VALIDATION_SPLIT = 0.2

IMG_HEIGHT = 224 # Example image height
IMG_WIDTH = 224 # Example image width
BATCH_SIZE = 32
VALIDATION_SPLIT = 0.2

rescale = tf.keras.layers.Rescaling(1./255) # Normalize pixel values
to [0, 1]

class_names = sorted(os.listdir(TRAIN_DIR))
if not class_names:
  print("No class directories found in the train folder!")
else:
  print(f"Found {len(class_names)} classes: {class_names}")
```

```
Found 6 classes: ['acai', 'cupuacu', 'graviola', 'guarana', 'pupunha',
'tucuma']
```

```
from PIL import Image, UnidentifiedImageError
corrupted_images = [] # List to store corrupted images path
# Loop through each class folder and check for corrupted images
for class_name in class_names:
  class_path = os.path.join(TRAIN_DIR, class_name) # Changed train_dir
to TRAIN_DIR
  if os.path.isdir(class_path): # Ensure it’s a valid directory
    images = os.listdir(class_path)
    for img_name in images:
      img_path = os.path.join(class_path, img_name)
      try:
        with Image.open(img_path) as img:
          img.verify() # Verify image integrity
      except (IOError, UnidentifiedImageError):
        corrupted_images.append(img_path)

# Print results
if corrupted_images:
  print("\nCorrupted Images Found:")
  for img in corrupted_images:
    print(img)
else:
  print("\nNo corrupted images found.")
```

```
No corrupted images found.
```

```
# Dictionary to store class counts
class_counts = {}
for class_name in class_names:
  class_path = os.path.join(TRAIN_DIR, class_name)
  if os.path.isdir(class_path):
    images = [img for img in os.listdir(class_path) if
img.lower().endswith(('.png', '.jpg', '.jpeg'))]
    class_counts[class_name] = len(images) # Count images in each
class
# Print Class Balance
print("\nClass Distribution:")
print("=" * 45)
print(f"{'Class Name':<25}{'Valid Image Count':>15}")
print("=" * 45)
for class_name, count in class_counts.items():
  print(f"{class_name:<25}{count:>15}")
print("=" * 45)
```

```
Class Distribution:
```

```
==============================================
Class Name                Valid Image Count
==============================================
acai                                     15
cupuacu                                  15
graviola                                 15
guarana                                  15
pupunha                                  15
tucuma                                   15
==============================================
```

```
import random
selected_images = [] # Store image paths
selected_labels = [] # Store corresponding class names
for class_name in class_names:
  class_path = os.path.join(TRAIN_DIR, class_name)
  if os.path.isdir(class_path):
    images =[img for img in os.listdir(class_path) if
img.lower().endswith(('.png', '.jpg', '.jpeg'))]
  if images: # Ensure the class folder is not empty
    selected_img = os.path.join(class_path, random.choice(images))
    selected_images.append(selected_img)
    selected_labels.append(class_name)
```

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
# Determine grid size
num_classes = len(selected_images)
cols = (num_classes + 1) // 2 # Determine columns
rows = 2 # Fixed rows for layout
fig, axes = plt.subplots(rows, cols, figsize=(12, 6))
for i, ax in enumerate(axes.flat):
  if i < num_classes:
    img = mpimg.imread(selected_images[i])
    ax.imshow(img)
    ax.set_title(selected_labels[i])
    ax.axis("off")
  else:
    ax.axis("off") # Hide empty subplots
plt.tight_layout()
plt.show()
```

```
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
TRAIN_DIR,
labels='inferred',
label_mode='int',
image_size=(IMG_HEIGHT, IMG_WIDTH),
interpolation='nearest',
batch_size=BATCH_SIZE,
shuffle=True,
validation_split=VALIDATION_SPLIT,
subset='training',
seed=123
)
train_ds = train_ds.map(lambda x, y: (rescale(x), y))
```

```
Found 90 files belonging to 6 classes.
Using 72 files for training.
```

```
# Create validation dataset with normalization
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
TRAIN_DIR,
labels='inferred',
label_mode='int',
image_size=(IMG_HEIGHT, IMG_WIDTH),
interpolation='nearest',
batch_size=BATCH_SIZE,
shuffle=False,
validation_split=VALIDATION_SPLIT,
subset='validation',
seed=123
```

```
)
val_ds = val_ds.map(lambda x, y: (rescale(x), y))
```

```
Found 90 files belonging to 6 classes.
Using 18 files for validation.
```

```
from tensorflow.keras.applications import VGG16

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

for layer in base_model.layers:
  layer.trainable = False



model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dense(1024, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
```

```
Model: "sequential_1"

┏━━━
```

```
 Non-trainable params: 14,714,688 (56.13 MB)
```

```
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

test_loss, test_acc = model.evaluate(val_ds)
print(f"Validation Accuracy: {test_acc:.2f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 8s 8s/step - accuracy: 0.0000e+00 - loss:
2.6608
Validation Accuracy: 0.00
```

```
 # Replace with your test data path

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    TEST_DIR,
    labels='inferred',
    label_mode='int',
    image_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    shuffle=False,
    interpolation='nearest',
    seed=123
)


test_ds = test_ds.map(lambda x, y: (rescale(x), y))
```

```
Found 30 files belonging to 6 classes.
```

```
test_loss, test_accuracy = model.evaluate(test_ds)
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Loss: {test_loss:.4f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 21s 21s/step - accuracy: 0.0667 - loss:
2.7847
Test Accuracy: 0.0667
Test Loss: 2.7847
```

Fine-Tuning

```
base_model = VGG16(
    weights='imagenet',
    include_top = False,
    input_shape=(224,224,3)
)

base_model.trainable = True
```

```
set_trainable = False

for layer in base_model.layers:
  if layer.name == 'block5_conv1':
    set_trainable = True
  if set_trainable:
    layer.trainable = True
  else:
    layer.trainable = False

for layer in base_model.layers:
  print(layer.name,layer.trainable)
```

```
input_layer_4 False
block1_conv1 False
block1_conv2 False
block1_pool False
block2_conv1 False
block2_conv2 False
block2_pool False
block3_conv1 False
block3_conv2 False
block3_conv3 False
block3_pool False
block4_conv1 False
block4_conv2 False
block4_conv3 False
block4_pool False
block5_conv1 True
block5_conv2 True
block5_conv3 True
block5_pool True
```

```
base_model.summary()
```

```
Model: "vgg16"

┏━━━
```

```
──────┤
│ block1_conv2 (Conv2D)            │ (None, 224, 224, 64)   │
36,928 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block1_pool (MaxPooling2D)       │ (None, 112, 112, 64)   │
0 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block2_conv1 (Conv2D)            │ (None, 112, 112, 128)  │
73,856 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block2_conv2 (Conv2D)            │ (None, 112, 112, 128)  │
147,584 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block2_pool (MaxPooling2D)       │ (None, 56, 56, 128)    │
0 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block3_conv1 (Conv2D)            │ (None, 56, 56, 256)    │
295,168 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block3_conv2 (Conv2D)            │ (None, 56, 56, 256)    │
590,080 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block3_conv3 (Conv2D)            │ (None, 56, 56, 256)    │
590,080 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block3_pool (MaxPooling2D)       │ (None, 28, 28, 256)    │
0 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block4_conv1 (Conv2D)            │ (None, 28, 28, 512)    │
1,180,160 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block4_conv2 (Conv2D)            │ (None, 28, 28, 512)    │
2,359,808 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ block4_conv3 (Conv2D)            │ (None, 28, 28, 512)    │
2,359,808 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
```

```
│ block4_pool (MaxPooling2D)       │ (None, 14, 14, 512)    │
0 │
├─────────────────────────────────┼────────────────────────┼────────────
──────┤
│ block5_conv1 (Conv2D)            │ (None, 14, 14, 512)    │
2,359,808 │
├─────────────────────────────────┼────────────────────────┼────────────
──────┤
│ block5_conv2 (Conv2D)            │ (None, 14, 14, 512)    │
2,359,808 │
├─────────────────────────────────┼────────────────────────┼────────────
──────┤
│ block5_conv3 (Conv2D)            │ (None, 14, 14, 512)    │
2,359,808 │
├─────────────────────────────────┼────────────────────────┼────────────
──────┤
│ block5_pool (MaxPooling2D)       │ (None, 7, 7, 512)      │
0 │
└─────────────────────────────────┴────────────────────────┴────────────
──────┘

 Total params: 14,714,688 (56.13 MB)

 Trainable params: 7,079,424 (27.01 MB)

 Non-trainable params: 7,635,264 (29.13 MB)
```

```
model = Sequential()

model.add(base_model)
model.add(Flatten())
model.add(Dense(256,activation='relu'))
model.add(Dense(10,activation='softmax'))

model.compile(optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_ds, validation_data=val_ds, epochs=10)
```

```
Epoch 1/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 63s 19s/step - accuracy: 0.0838 - loss:
3.0150 - val_accuracy: 0.0556 - val_loss: 2.9432
Epoch 2/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 58s 19s/step - accuracy: 0.3889 - loss:
2.0300 - val_accuracy: 0.1111 - val_loss: 2.1587
Epoch 3/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 82s 18s/step - accuracy: 0.6250 - loss:
1.5533 - val_accuracy: 0.3889 - val_loss: 1.7913
Epoch 4/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 58s 18s/step - accuracy: 0.7109 - loss:
1.2940 - val_accuracy: 0.6111 - val_loss: 1.4704
```

```
Epoch 5/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 60s 19s/step - accuracy: 0.8976 - loss:
1.0038 - val_accuracy: 0.7222 - val_loss: 1.2760
Epoch 6/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 82s 19s/step - accuracy: 0.9518 - loss:
0.8194 - val_accuracy: 0.8333 - val_loss: 0.9393
Epoch 7/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 80s 18s/step - accuracy: 0.9705 - loss:
0.6029 - val_accuracy: 0.8889 - val_loss: 0.7386
Epoch 8/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 58s 18s/step - accuracy: 1.0000 - loss:
0.5067 - val_accuracy: 0.8333 - val_loss: 0.8006
Epoch 9/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 58s 18s/step - accuracy: 1.0000 - loss:
0.3823 - val_accuracy: 0.8889 - val_loss: 0.5668
Epoch 10/10
3/3 ━━━━━━━━━━━━━━━━━━━━ 92s 23s/step - accuracy: 1.0000 - loss:
0.3140 - val_accuracy: 0.8889 - val_loss: 0.5414
```

```
<keras.src.callbacks.history.History at 0x7e0d1c2e6850>
```

```
test_loss, test_acc = model.evaluate(val_ds)
print(f"Validation Accuracy: {test_acc:.2f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 7s 7s/step - accuracy: 0.0000e+00 - loss:
2.7137
Validation Accuracy: 0.00
```

```
  # Replace with your test data path

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    TEST_DIR,
    labels='inferred',
    label_mode='int',
    image_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    shuffle=False,
    interpolation='nearest',
    seed=123
)


test_ds = test_ds.map(lambda x, y: (rescale(x), y))
```

```
Found 30 files belonging to 6 classes.
```

```
test_loss, test_accuracy = model.evaluate(test_ds)
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Loss: {test_loss:.4f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 12s 12s/step - accuracy: 0.1000 - loss:
2.7353
Test Accuracy: 0.1000
Test Loss: 2.7353
```